**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# LESSON 023 · Hash tables with arrays

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. In a binary search tree, where are values smaller and larger than a node stored?

Q2. What does -1 represent in our pointer arrays?

Q3. What is a collision in hashing?

Q4. For table size 5, what bucket does key 17 map to using key MOD 5?

**Do Now** Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# I Do · Read new knowledge

## Key vocabulary

Recall the term — hash table

Recall the term — key

Recall the term — hash function

Recall the term — bucket

Recall the term — collision

Recall the term — chaining

Recall the term — lookup

## New learning

Create fixed buckets — CODE:
int[,] table = new int[5,3];
int[] count = new int[5]; — Each row is a bucket with three available collision slots.

Hash — CODE:
int bucket = key % 5; — The same key maps to the same row.

Add — CODE:
table[bucket, count[bucket]] = key;
count[bucket]++; — A collision goes into the next free column in that bucket.

Search/remove — CODE:
for (int c = 0; c < count[bucket]; c++)
{
if (table[bucket,c] == key)
{
// found
}
} — Only the calculated bucket is searched. Removal can shift later bucket items left.

## Worked example

Add 12, 17 and 22

1. 12 — CODE:
table[2,0] = 12
count[2] = 1

2. 17 — CODE:
table[2,1] = 17

count[2] = 2

3. 22 — CODE:
table[2,2] = 22
count[2] = 3

4. 17 — CODE:
hash → bucket 2
scan row 2
17 found at column 1

## We Do · Practise with guidance

Calculate hash

Table size 10; key 37.

Complete the same calculation

```
37 MOD 10 = ?
```

```
37 MOD 10 = 7
```

Add key

Use bucket and count arrays.

Fill the same code

```
int bucket = key % ?;
table[bucket, ?] = key;
count[bucket]++;
```

```
int bucket = key % 5;
table[bucket, count[bucket]] = key;
count[bucket]++;
```

Collision

12 and 17 both map to bucket 2.

Complete the bucket

```
bucket 2:
column 0 = 12
column 1 = ?
```

```
bucket 2:
column 0 = 12
column 1 = 17
```

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Remove from bucket

Remove column 1 when count is 3.

Complete the shift



RHS COMPUTING / OCR H446 / 1.4.2 LESSON 023 · DRAFT

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

```
bool found = false;
for (int i = 0; i < counts[b]; i++)
{
if (buckets[b,i] == key)
found = true;
}
Console.WriteLine(found); // True
```

Q4. Find the key in its bucket; shift subsequent entries to close the gap.
CODE:

```
int key = 17;
int b = key % 5;
for (int i = 0; i < counts[b]; i++)
{
if (buckets[b,i] == key)
{
for (int j = i; j < counts[b]-1; j++)
buckets[b,j] = buckets[b,j+1];
counts[b]--;
break;
}
}
```

Bucket 2 becomes [12, 22, —], count = 2. Searching for 22 still succeeds.

### Exit Ticket · Model answer

True. Different keys can hash to the same bucket. This is called a collision.

## Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**



RHS COMPUTING / OCR H446 / 1.4.2 LESSON 023 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Create buckets
Create a 5×3 integer table and a count array.

### Question 2

Add keys with collisions
Add 12, 17, 22 and 9 using key MOD 5.

### Question 3

Search only the correct bucket
Search for key 17 using its hash value and a loop across that bucket.

### Question 4

Remove 17
Find 17 in its bucket, shift later values left, and reduce the bucket count.

## Exit Ticket · Answer before checking

A hash function tells you which bucket to search; it does not guarantee that only one key will use that bucket.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.





**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

## Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. Smaller values go left and larger values go right.

Q2. null / no child.

Q3. Two keys map to the same hash location.

Q4. 2.

### You Do pages · Independent questions

Q1. Create a table with 5 rows (buckets) and capacity 3 per bucket. Counts begin at zero.
CODE:
int[,] buckets = new int[5,3];
int[] counts = new int[5];
Use counts[b] as the next free column in bucket b.

Q2. Hash each key using key MOD 5.
CODE:
int[] keys = {12,17,22,9};
foreach (int key in keys)
{
int b = key % 5;
buckets[b,counts[b]] = key;
counts[b]++;
}
12 MOD 5 = 2; 17 MOD 5 = 2; 22 MOD 5 = 2; 9 MOD 5 = 4.
CODE:
Bucket 2: [12, 17, 22]
Bucket 4: [ 9, -, -]
Collision resolution: keys sharing a hash occupy successive columns.

Q3. Hash 17 first, then search only bucket 2.
CODE:
int key = 17;
int b = key % 5;